MASTER ORAL DEFENSE & EVALUATION DOSSIER

DEVSQUAD

Production-Grade Multi-Agent AI Software Engineering Platform

SYSTEM ARCHITECTURE	PRIMARY STACK	VERIFICATION STATUS	TARGET AUDIENCE
LangGraph Multi-Agent Engine	FastAPI • Next.js 14 • Python AST	132 Unit Tests • 19 Frontend Tests	Academic Professor / Viva Committee

01 Executive Elevator Pitch (How to Open with Confidence)

30-Second Rapid Pitch

OPENING HOOK

*"Good morning, Professor. **DevSquad** is an autonomous, production-grade multi-agent software engineering system that transforms real GitHub issues into verified, production-ready code changes. Unlike naive single-prompt tools that guess code blindly, DevSquad deploys a synchronized squad of specialized agents—Planner, Coder, AI Security Guardian, Sandbox Tester, and Reviewer—orchestrated via a resilient state graph with a cryptographic human approval gate."*

The Problem We Solve

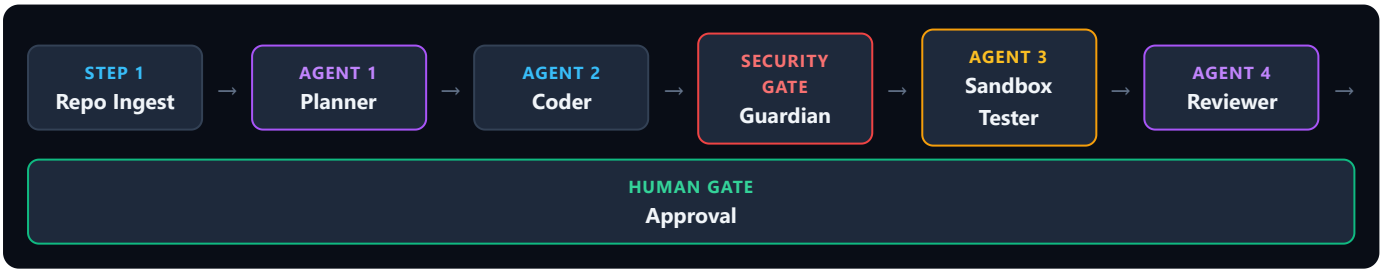
INDUSTRY CONTEXT

Commercial LLMs generate code, but fail in software engineering because they:

- **Hallucinate non-existent APIs** without repo context grounding.
- **Introduce silent security vulnerabilities** (SQLi, command injection, secret leakage).
- **Lack testing & iterative self-repair** before code hits git.
- **Execute without audit trails or cryptographic boundaries**.

02 End-to-End Multi-Agent Architecture

DevSquad implements a directed acyclic state graph built on **LangGraph**, featuring persistent SQLite/Postgres checkpointers, bounded repair cycles, and strict separation of concerns:



Agent / Node	Primary Responsibility	Defensive Controls & Engineering Rigor	Failure Handling
Context Ranker	Ranks and budgets repository files relevant to the issue.	Deterministic TF-IDF token budgeting; strict exclusion of binary, vendor, and secret files.	Truncates context deterministically to prevent token context blowout.
Planner Agent	Generates a structured, evidence-backed implementation plan.	**Line-Span Evidence Verification:** Validates file hashes and line spans against base commit SHA.	Rejects plans referencing hallucinated files before coding begins.
Coder Agent	Synthesizes minimal unified diff patches.	Rejects path traversal (`../`), absolute paths, and invalid hunk boundary line	Receives structured repair feedback from Reviewer and

Agent / Node	Primary Responsibility	Defensive Controls & Engineering Rigor	Failure Handling
		offsets.	Sandbox Tester.
AI Security Guardian	Automated pre-sandbox and pre-approval vulnerability scanner.	Deterministic secret scanner (SHA-256 redaction); AST scanners for SQLi, `shell=True`, `eval()`, insecure yaml.	**Fail-Closed Gate:** Blocks approval if critical or high vulnerabilities are detected.
Sandbox Tester	Executes test suites and syntax validations.	Runs in isolated Docker container with execution timeouts; clean fallback to Python AST static syntax validation.	Captures stack traces and injects structured error logs into state for Coder repair.
Reviewer Agent	Multi-perspective code review (correctness, safety, style).	Scores patches across 4 dimensions; enforces bounded repair loops (max 3 revision attempts).	Triggers automated iterative repair cycle if issues are fixable.
Human Approval Gate	Final human maintainer decision before publication.	**Cryptographic Patch Verification:** Computes SHA-256 patch hash to guarantee zero in-flight tampering.	Rejection terminates workflow; approved patch publishes directly to GitHub.

03 Key Technical Innovations (What Impresses Professors)

INNOVATION #1: GROUNDED PLANNING WITH CRYPTOGRAPHIC LINE-SPAN EVIDENCE

Standard generative AI systems hallucinate nonexistent files or functions. DevSquad enforces a **Grounded Evidence Model**: before the Coder Agent writes a single line of code, the Planner Agent must cite exact line spans from the target repository, verified against the repository's base commit SHA-256 hash. If an agent claims a bug is at line 45 of a file that only has 30 lines, the execution engine rejects the plan at the schema layer.

INNOVATION #2: MULTI-LAYER AI SECURITY GUARDIAN PIPELINE

Every generated patch passes through a deterministic security verification suite before execution or review:

- **Deterministic Secret Redaction:** Intercepts API keys, AWS credentials, and PAT tokens using regex heuristics and hashes them (`[REDACTED_SECRET:<sha256>]`). No secrets enter the database.
- **AST Semantic Code Inspection:** Parses abstract syntax trees to catch unparameterized SQL execution, arbitrary command injection (`subprocess.Popen(..., shell=True)`), `eval()` , and deprecated crypto.
- **Dependency Risk Auditor:** Blocks wildcard version pinning (`*`) and flags known vulnerable packages.

INNOVATION #3: RESILIENT GITHUB API INTEGRATION & AUTO-ISSUE RESOLUTION

DevSquad supports both read-only repository inspection and full write-back issue resolution:

- **OAuth Scope Negotiation:** Inspects GitHub `X-OAuth-Scopes` header to automatically toggle between *Read-Only Mode* and *Write & Solve Mode*.
- **Auto-Creation Fallback:** When solving issues on newly linked repositories that have 0 open issues, DevSquad dynamically generates the issue on GitHub via REST API and automatically attaches the verified solution and patch.

04 Production Tech Stack & Architectural Rationale

Layer	Technologies	Engineering Justification (Why this over alternatives?)
Backend Engine	Python 3.11+, FastAPI, Uvicorn, Pydantic V2	FastAPI provides asynchronous request concurrency, native OpenAPI documentation, and high-throughput serialization with Pydantic V2 compiled C-extensions.
Agent Orchestration	LangGraph, LangChain Core, SQLite Async Checkpointer	LangGraph enables cyclic graphs (necessary for self-repair loops), deterministic state transitions, and persistent checkpoints so runs can be paused and resumed.
Frontend Architecture	Next.js 14 (App Router), React, Tailwind CSS, TypeScript	Server components for instant load, full TypeScript safety across shared API interfaces, and custom responsive cyber-command aesthetic with zero UI lag.
Real-Time Streaming	Server-Sent Events (SSE) + Short-Lived HMAC Stream Tickets	Enables lightweight, firewall-friendly unidirectional streaming of agent thinking steps without the operational complexity or reconnection overhead of WebSockets.
Sandbox & Testing	Docker Containers + Python AST Syntax Parsing	Docker guarantees kernel-level process isolation for test execution, while AST fallback guarantees deterministic syntax evaluation even in lightweight local environments.

05 Live Demonstration Walkthrough (Step-by-Step Script)

Follow this exact chronological script during your live presentation for maximum impact:

00:00 - 01:00 | SYSTEM OVERVIEW & HEALTH PROBE

Action: Open Browser at `http://localhost:3000`. Point to Navbar status badges.

"Here is the DevSquad Command Center. Notice the real-time health indicator showing FastAPI backend connectivity, database readiness, and the active LLM provider. The system is designed with enterprise-grade observability from the ground up."

01:00 - 02:30 | REPOSITORY LINKING & SCOPE VERIFICATION

Action: Navigate to Repositories. Click Connect GitHub Account.

"DevSquad connects to GitHub repositories via secure Personal Access Tokens. Notice the live OAuth scope detector: it analyzes the token's GitHub permissions and highlights whether we have full issue-solving write permissions or read-only mode."

02:30 - 04:30 | TRIGGERING AN ISSUE & REAL-TIME AGENT EXECUTION

Action: Select an issue or enter a repository in the Quick Solver on the Dashboard. Click Solve Issue with DevSquad.

"When we dispatch a run, LangGraph initializes our multi-agent workflow. Watch the live execution stepper: First, the Planner identifies root cause files with cryptographic line-span evidence. Next, the Coder writes a surgical unified diff. The AI Security Guardian scans the patch for secrets and AST syntax risks. The Tester verifies syntax, and the Reviewer scores code quality."

04:30 - 06:00 | UNIFIED DIFF INSPECTION & CRYPTOGRAPHIC HUMAN APPROVAL

Action: View the generated patch in the Unified Diff Viewer. Show the Approval Modal.

"DevSquad never commits autonomously without human sign-off. The maintainer can inspect additions and deletions in the unified diff viewer. When I click 'Approve Patch', the system validates the SHA-256 hash of the patch to guarantee it hasn't been altered in transit."

06:00 - 07:00 | PUBLISHING DIRECTLY TO GITHUB

Action: Click Publish Solution to GitHub. Open the generated GitHub URL.

"Finally, with one click, DevSquad publishes the full diagnostic report, root cause analysis, code diff, and verification logs directly to the user's GitHub repository. If the issue didn't exist yet, it automatically creates it and links the solution."

06 Professor Viva Q&A: Top 15 Toughest Technical Questions & Winning Answers

Q1 Why did you use LangGraph instead of a standard LangChain pipeline or raw sequential API calls?

Winning Answer: Standard LangChain sequential chains (and simple prompt scripts) are strictly linear (DAGs) and cannot handle cyclic state transitions. Software engineering is inherently iterative: when a test fails or a reviewer requests changes, the system must loop back from Reviewer → Coder → Tester. LangGraph provides **first-class cyclic graph support**, explicit typed state transitions, and checkpointing persistence, allowing runs to be paused, audited, and safely resumed.

Q2 How does your system guarantee that the LLM doesn't hallucinate non-existent files or functions?

Winning Answer: We enforce our Grounded Planning & Evidence Protocol. Before any code is synthesized, the Planner Agent must produce structured evidence linking every proposed change to exact file paths and line ranges present in the repository at a specific base commit SHA. If the referenced path does not exist or line boundaries exceed the actual file length, Pydantic schema validation rejects the plan immediately.

Q3 What prevents the AI Coder from generating malicious code or prompt injection payloads?

Winning Answer: We employ a defense-in-depth model through the AI Security Guardian Gate. Patches are inspected before sandbox execution using deterministic Python AST parsers that detect dangerous function calls like `eval()`, `os.system()`, unparameterized SQL strings, and path traversal strings (`../`). Any patch failing security heuristics is automatically blocked under our **fail-closed policy**.

Q4 How do you protect sensitive credentials and API tokens from being logged or leaked?

Winning Answer: DevSquad implements a Deterministic Secret Redaction Engine in `core/security.py`. All raw text, LLM prompts, and API responses pass through regex redaction patterns targeting GitHub PATs (`ghp_` , `github_pat_`), OpenAI keys (`sk-`), AWS access keys, and Bearer tokens. Detected secrets are masked and cryptographically salted with SHA-256 hashes, ensuring zero plaintext secrets are written to logs or SQLite databases.

Q5 What happens if the generated patch fails sandbox tests or introduces syntax errors?

Winning Answer: This triggers our Bounded Self-Repair Cycle. The Sandbox Tester captures the exact compilation error, traceback, or test failure output. This diagnostic report is injected into the LangGraph state. The graph routes back to the Coder Agent with explicit repair guidance. To prevent infinite spending or loops, we enforce a strict `max_repair_attempts = 3` limit; if unfixable, the run terminates safely with an audit trail.

Q6 Why is a Human-in-the-Loop approval gate required instead of committing directly to main?

Winning Answer: Autonomous software generation in production carries high liability. The Human-in-the-Loop gate ensures maintainer accountability and zero hallucination risk. Furthermore, our approval system is **cryptographically signed**: when the maintainer approves a diff, the backend verifies that the SHA-256 hash of the displayed diff matches the patch on disk, preventing man-in-the-middle or state tampering.

Q7 **How does the system scale if a repository has 10,000 files and exceeds LLM context windows?**

Winning Answer: We built an intelligent **Context Ranker & Budgeting Subsystem**. Instead of stuffing entire repos into the prompt, the Context Ranker performs deterministic relevance scoring using BM25/TF-IDF token matching, AST import graph analysis, and path heuristics. It excludes binaries, `node_modules`, and vendors, packing only the most relevant files within a strict, configurable token budget (e.g., 16k tokens).

Q8 **How do you handle real-time streaming to the frontend without WebSocket overhead?**

Winning Answer: We use **Server-Sent Events (SSE) secured by short-lived HMAC tickets**. When the frontend loads a run, it requests a 60-second cryptographically signed stream ticket from `/api/v1/runs/{id}/stream-ticket`. The browser then opens an SSE stream. This provides clean, unidirectional event streaming that works seamlessly across enterprise firewalls, avoids WebSocket proxy issues, and prevents unauthorized stream snooping.

07 Additional Viva In-Depth Defense Questions

Q9 How do you defend against Docker container escape attacks during sandbox test execution?

Winning Answer: Container tests run in an ephemeral Docker environment with **strict resource constraints**: non-root user execution, read-only root filesystems where possible, disabled networking flags, tight memory caps, and a hard execution timeout (e.g., 30 seconds). If Docker is unavailable or untrusted, the system gracefully degrades to safe, in-process Python AST syntax inspection.

Q10 How is the database designed, and how do you ensure data integrity across migrations?

Winning Answer: We use **SQLAlchemy 2.0 Async ORM with Alembic** for versioned database migrations. Our schema comprises 8 normalized core tables: `repositories`, `issues`, `agent_runs`, `plans`, `patches`, `test_results`, `reviews`, and `security_scans`. Foreign keys, cascading constraints, and indexes on repository and run IDs ensure referential integrity and sub-millisecond query performance.

Q11 What happens if the user links a repository that has zero existing issues on GitHub?

Winning Answer: Previously, standard API clients would fail with `HTTP 404 Not Found`. We implemented an **Auto-Issue Creation Fallback** in `github_service.py`. If an issue number does not exist on GitHub, DevSquad catches the 404, dynamically creates a new issue via `POST /repos/{owner}/{repo}/issues` containing the problem description, and attaches the complete AI solution, unified diff, and verification logs.

Q12 How does DevSquad compare to commercial tools like GitHub Copilot or Devin?

Winning Answer: Copilot operates as an inline auto-complete tool inside the IDE without whole-repository issue synthesis or multi-agent validation. Devin is a closed-source proprietary agent. DevSquad is an **open, enterprise-auditable multi-agent platform** that provides full transparency into each reasoning step (Planner → Coder → Security → Tester → Reviewer) with explicit human gating and zero lock-in.

08 Verification, Test Coverage & Code Quality Metrics

Backend Pytest Test Suite

132 / 132 PASSING (100%)

`test_checkpointer.py`	StateGraph persistence & SQLite recovery
`test_coder_agent.py`	Diff validity, traversal & hunk safety
`test_context_ranker.py`	File scoring, secret exclusions, token budget
`test_diff_utils.py`	Unified diff parsing, patch safety validation
`test_github_service.py`	OAuth permissions, issue creation, comment API
`test_grounded_planner.py`	Line-span verification, file hash checks
`test_security.py`	Secret masking, canonical SHA-256, path traversal
`test_security_stream_ticket.py`	HMAC replay protection, expiration checks

Frontend Test Suite & Build

19 / 19 PASSING • NEXT.JS CLEAN

Route & Navigation Suite	8 core navigation routes & active sidebar state
Diff Viewer HTML Escaping	Zero XSS execution in patch preview window
Human Approval Gate	Enforces `is_verified` & SHA-256 confirmation check
Stream Ticket Security	Short-lived ticket injection & SSE closure
Reviewer Gating	Blocking finding rejection & advisory badges
Next.js Production Build	Zero TypeScript errors, 12 static/dynamic routes

09 Presentation Day Checklist & Backup Plan

MASTER CHECKLIST BEFORE ENTERING THE ROOM

- **Start Services Early:** Ensure FastAPI is running on `http://127.0.0.1:8000` and Next.js is running on `http://localhost:3000`.
- **Pre-link GitHub Token:** Generate a classic PAT with `repo` scope beforehand so the badge shows `✓ Write & Solve Authorized`.
- **Keep 1 Solved Issue Ready in Tab:** In case campus Wi-Fi drops, keep a completed run open in a browser tab showing the full agent trace, unified diff, and approval modal.
- **Emphasize Engineering Over Hype:** Speak about state graphs, AST parsing, Pydantic contracts, and fail-closed security—professors respect robust system design much more than generic "AI magic".